Unit 2 - Data compression

Essential reading

Wayner, Peter *Compression Algorithms for Real Programmers*. (Morgan Kaufmann, 2000) [ISBN 0-12-788774-1]. Chapter 1.

Further reading

Salomon, David *A Guide to Data Compression Methods*. (Springer, 2001) [ISBN 0-387-95260-8]. Introduction.

Importance of data compression

Data compression techniques is motivated mainly by the need to improve efficiency of information processing. This includes improving the following main aspects in the digital domain:

- storage efficiency
- efficient usage of transmission bandwidth
- reduction of transmission time.

Although the cost of storage and transmission bandwidth for digital data have dropped dramatically, the demand for increasing their capacity in many applications has been growing rapidly ever since. There are cases in which extra storage or extra bandwidth is difficult to achieve, if not impossible.
Data compression as a means may make much more efficient use of existing resources with less cost. Active research on data compression can lead to innovative new products and help provide better services.

Brief history

Data compression can be viewed as the art of creating shorthand representations for the data even today, but this process started as early as 1,000 BC. The short list below gives a brief survey of the historical milestones:

- 1000BC, Shorthand
- 1829, Braille code
- 1843, Morse code
- 1930 onwards, Analog compression
- 1950, Huffman codes
- 1975, Arithmetic coding
- 1977, Dictionary-based compression
- 1980s
  - early 80s, FAX
  - mid-80s, Video conferencing, still images (JPEG), improved FAX standard (JBIG)
  - late 80s, onward Motion video compression (MPEG)
- 1990s
  - early 90s, Disk compression (stacker)
  - mid-90s, Satellite TV
  - late 90s, Digital TV (HDTV), DVD, MP3

- 2000s Digital TV (HDTV), DVD, MP3

Source data

In this subject guide, the word *data* includes any digital information that can be processed in a computer, which includes text, voice, video, still images, audio and movies. The data before any compression (i.e. encoding) process is called the *source data*, or the *source* for short.

Three common types of source data in the computer are *text* and (digital) *image* and *sound*.

- **Text** data is usually represented by ASCII code (or EBCDIC).
- **Image** data is represented often by a two-dimensional array of *pixels* in which each pixel is associated with its color code.
- **Sound** data is represented by a wave (periodic) function.

In the application world, the source data to be compressed is likely to be so-called *multimedia* and can be a mixture of text, image and sound.

Lossless and lossy data compression

Data compression is simply a means for efficient digital representation of a source of data such as text, image and the sound. The goal of data compression is to represent a source in digital form with as few bits as possible while meeting the minimum requirement of reconstruction. This goal is achieved by removing any redundancy presented in the source.

There are two major families of compression techniques in terms of the possibility of reconstructing the original source. They are called *Lossless* and *lossy* compression.

Lossless compression

A compression approach is lossless only if it is possible to exactly reconstruct the original data from the compressed version. There is no loss of any information during the compression[1] process.

[1] *This, when used as a general term, actually includes both compression and decompression process.*

Lossless compression techniques are mostly applied to symbolic data such as character text, numeric data, computer source code and executable graphics and icons.

Lossless compression techniques are also used when the original data of a source are so important that we cannot afford to lose any details. For example, medical images, text and images preserved for legal reasons; some computer executable files, etc.

Lossy compression

A compression method is lossy compression only if it is not possible to reconstruct the original exactly from the compressed version. There are some insignificant details that may get lost during the process of compression.

Approximate reconstruction may be very good in terms of the compression-ratio but usually it often requires a trade-off between the visual quality and the computation complexity (i.e. speed).

Data such as multimedia images, video and audio are more easily compressed

by lossy compression techniques.

Main compression techniques

Data compression is often called *coding* due to the fact that its aim is to find a specific *short* (or shorter) way of representing data. *Encoding* and *decoding* are used to mean compression and decompression respectively. We outline some major compression algorithms below:

- Run-length coding
- Quantisation
- Statistical coding
- Dictionary-based coding
- Transform-based coding
- Motion prediction.

Run-length coding

The idea of Run-length coding is to replace consecutively repeated symbols in a source with a code pair which consists of either the repeating symbol and the number of its occurrences, or sequence of non-repeating symbols.

Example 1.1 *String* ABBBBBBBCC *can be represented by* Ar_7Br_2C, *where* r_7 *and* r_2 *means 7 and 2 occurrences respectively.*

All the symbols are represented by an 8-bit ASCII codeword.

Quantisation

The basic idea of quantisation is to apply a certain computation to a set of data in order to achieve an approximation in a simpler form.

Example 1.2 *Consider storing a set of integers (7, 223, 15, 28, 64, 37, 145). Let* x *be an integer in the set. We have* $7 \leq x \leq 223$. *Since* $0 < x < 255$ *and* $2^8 = 256$, *it needs 8 binary bits to represent each integer above.*

However, if we use a multiple, say 16, as a common divider to apply to each integer and round its value to the nearest integer, the above set becomes (0, 14, 1, 2, 4, 2, 9) after applying the computation x *div 16. Now each integer can be stored in 4 bits, since the maximum number 14 is less than* $2^4 = 16$.

Statistical coding

The idea of statistical coding is to use statistical information to replace a fixed-size code of symbols by a, hopefully, shorter variable-sized code.

Example 1.3 *We can code the more frequently occurring symbols with fewer bits. The statistical information can be obtained by simply counting the frequency of each character in a file. Alternatively, we can simply use the probability of each character.*

Dictionary-based coding

The dictionary approach consists of the following main steps:

1. read the file
2. find the frequently occurring sequences of symbols (FOSSs)
3. build up a dictionary of these FOSSs
4. associate each sequence with an index (usually a fixed length code)
5. replace the FOSS occurrences with the indices.

Transform-based coding

The transform-based approach models data by mathematical functions, usually by periodic functions such as $cos(x)$ and applies mathematical rules to primarily diffuse data. The idea is to change a mathematical quantity such as a sequence of numbers[2] to another form with useful features. It is used mainly in lossy compression algorithms involving the following activities:

[2]... *or anything else*

- analysing the signal (sound, picture etc.)
- decomposing it into frequency components
- making use of the limitations of human perception.

Motion prediction

Again, motion prediction techniques are lossy compression for sound and moving images.

Here we replace objects (say, an 8×8 block of pixels) in frames with references to the same object (at a slightly different position) in the previous frame.

Compression problems

In this course, we view data compression as algorithmic problems. We are mainly interested in compression algorithms for various types of data.

There are two classes of compression problems of interest (Davisson and Gray 1976):

- **Distortion-rate problem** Given a constraint on transmitted data rate or storage capacity, the problem is to compress the source at, or below, this rate but at the highest fidelity possible.
  Compression in areas of voice mail, digital cellular mobile radio and video conferencing are examples of the distortion-rate problems.
- **Rate-distortion problem** Given the requirement to achieve a certain pre-specified fidelity, the problem is to meet the requirements with as few bits per second as possible.
  Compression in areas of CD-quality audio and motion-picture-quality video are examples of rate-distortion problems.

Algorithmic solutions

In areas of data compression studies, we essentially need to analyse the characteristics of the data to be compressed and hope to deduce some patterns in order to achieve a compact representation. This gives rise to a variety of data modelling and representation techniques, which are at the heart of compression techniques. Therefore, there is no 'one size fits all' solution for data compression problems.

Compression and decompression

Due to the nature of data compression, any compression algorithm will not work unless a decompression approach is also provided. We may use the term *compression algorithm* to actually mean both compression algorithm and the decompression algorithm. In this subject, we sometimes do not discuss the decompression algorithm when the decompression process is obvious or can be easily derived from the compression process. However, you should always make sure that you know the decompression solutions.

In many cases, the efficiency of the decompression algorithm is of more concern than that of the compression algorithm. For example, movies, photos, and audio data are often compressed once by the artist and then decompressed many times by millions of viewers. However, the efficiency of compression is sometimes more important. For example, programs may record audio or video files directly to computer storage.

Compression performance

The performance of a compression algorithm can be measured by various criteria. It depends on what is our priority concern. In this subject guide, we are mainly concerned with the effect that a compression makes (i.e. the difference in size of the input file before the compression and the size of the output after the compression).

It is difficult to measure the performance of a compression algorithm in general because its compression behaviour depends much on whether the data contains the right patterns that the algorithm looks for.

The easiest way to measure the effect of a compression is to use the compression ratio.

The aim is to measure the effect of a compression by the shrinkage of the size of the source in comparison with the size of the compressed version.

There are several ways of measuring the compression effect:

- **Compression ratio**. This is simply the ratio of size.after.compression to size.before.compression or

$$\text{Compression ratio} = \frac{\text{size.after.compression}}{\text{size.before.compression}}$$

- **Compression factor**. This is the reverse of *compression ratio*.

$$\text{Compression factor} = \frac{\text{size.before.compression}}{\text{size.after.compression}}$$

- **Saving percentage**. This shows the shrinkage as a percentage.

$$\text{Saving percentage} = \frac{\text{size.before.compression} - \text{size.after.compression}}{\text{size.before.compression}}\%$$

Note: some books (e.g. Sayood(2000)) defines the compression ratio as our compression factor.

Example 1.4 *A source image file (pixels* 256×256*) with 65,536 bytes is compressed into a file with 16,384 bytes. The compression ratio is 1/4 and the compression factor is 4. The saving percentage is: 75%*

In addition, the following criteria are normally of concern to the programmers:

- **Overhead**. Overhead is some amount of extra data added into the compressed version of the data. The overhead can be large sometimes although it is often much smaller than the space saved by compression.
- **Efficiency** This can be adopted from well established algorithm analysis techniques. For example, we use the big-O notation for the time efficiency and the storage requirement. However, compression algorithms' behaviour can be very inconsistent but it is possible to use past empirical results.
- **Compression time** We normally consider the time for encoding and for decoding separately. In some applications, the decoding time is more important than encoding. In other applications, they are equally important.
- **Entropy**[3]. If the compression algorithm is based on statistical results, then entropy can be used to help make a useful judgement.

[3] *We shall introduce the concept of entropy later*

Limits on lossless compression

How far can we go with a lossless compression? What is the best compression we can achieve in a general case? The following two statements may slightly surprise you:

1. No algorithm can compress all (possible) files, even by one byte.
2. No algorithm can compress even 1% of all (possible) files even by one byte.

An informal reasoning for the above statements can be found below:

1. Consider the compression of a `big.file` by a lossless compression algorithm called `cmpres`. If statement 1 were not true, we could then effectively repeat the compression process to the source file.

   By 'effectively', we mean that the compression ratio is always < 1. This means that the size of the compressed file is reduced every time when running programme `cmpres`. So `cmpres(cmpres(cmpres(`$\cdots$ `cmpres(`big.file`)`$\cdots$ `)))`, the output file after compression many times, would be of size 0.

   Now it would be impossible to losslessly reconstruct the original.

2. Compressing a file can be viewed as mapping the file to a different (hopefully shorter) file.

   Compressing a file of n bytes (in size) by at least 1 byte means mapping the file of n bytes to a file of $n-1$ bytes or fewer bytes. There are $(2^8)^n = 256^n$ files of n bytes and 256^{n-1} of $n-1$ bytes in total. This means that the proportion of the successful 1-to-1 mappings is only $256^{n-1}/256^n = 1/256$ which is less than 1%.

Learning outcomes

On completion of your studies in this chapter, you should be able to:

- outline the brief history of Data compression

- explain how to distinguish lossless data compression from lossy data compression
- outline the main compression approaches
- measure the effect and efficiency of a data compression algorithm
- explain the limits of lossless compression.

Activities

1. Investigate what compression software is available on your computer system.

2. Suppose that you have compressed a file myfile using a compression utility available on your computer. What is the name of the compressed file?

3. Use a compression facility on your computer system to compress a text file called myfile containing the following text:

   This is a test.

   Suppose you get a compressed file called myfile.gz after compression. How would you measure the size of myfile and of myfile.gz?

4. Suppose the size of myfile.gz is 20 KB while the original file myfile is of size 40 KB. Compute the compression ratio, compression factor and saving percentage.

5. A compression process is often said to be 'negative' if its compression ratio is greater than 1.

   Explain why negative compression is an inevitable consequence of a lossless compression.

Laboratory

1. If you have access to a computer using Unix or Linux operating system, can you use compress or gzip command to compress a file?

2. If you have access to a PC with Windows, can you use WinZip to compress a file?

3. How would you recover your original file from a compressed file?

4. Can you use uncompress or gunzip command to recover the original file?

5. Implement a method compressionRatio in Java which takes two integer arguments sizeBeforeCompression and sizeAfterCompression and returns the compression ratio. See Activity 4 for example.

6. Similarly, implement a method savingPercentage in Java which takes two integer arguments sizeBeforeCompression and sizeAfterCompression and returns the saving percentage.

Sample examination questions

1. Explain briefly the meanings of *lossless* compression and *lossy* compression. For each type of compression, give an example of an application, explaining why it is appropriate.

2. Explain why the following statements are considered to be true in describing the absolute limits on lossless compression.

   - No algorithm can compress all files, even by one byte.
   - No algorithm can compress even 1% of all files, even by one byte.